Class 07: Machine Learning 01

Ryan Ellis (PID: A17673864)

Table of contents

Background

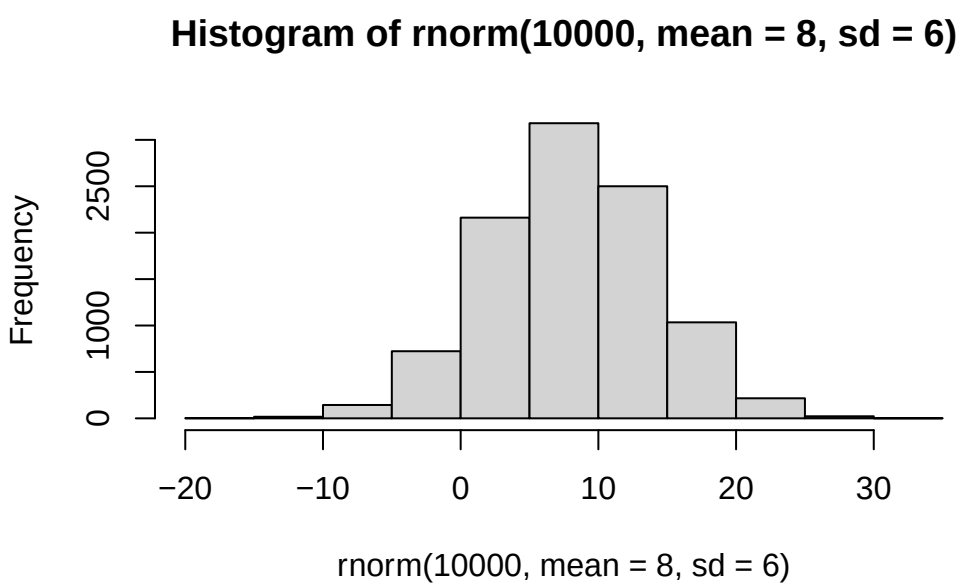
Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionallity reduction**.

K-means clustering

The main function in "base" R for K-means clustering is called `K-means()`

Before we go to deep, lets use something simple to help us figure out its function and usage. To accomplish this we use the `rnorm()` function:

```
hist(rnorm(10000,mean=8,sd=6))
```




```
x <- c( rnorm(30,-3),rnorm(30,3))
x
```

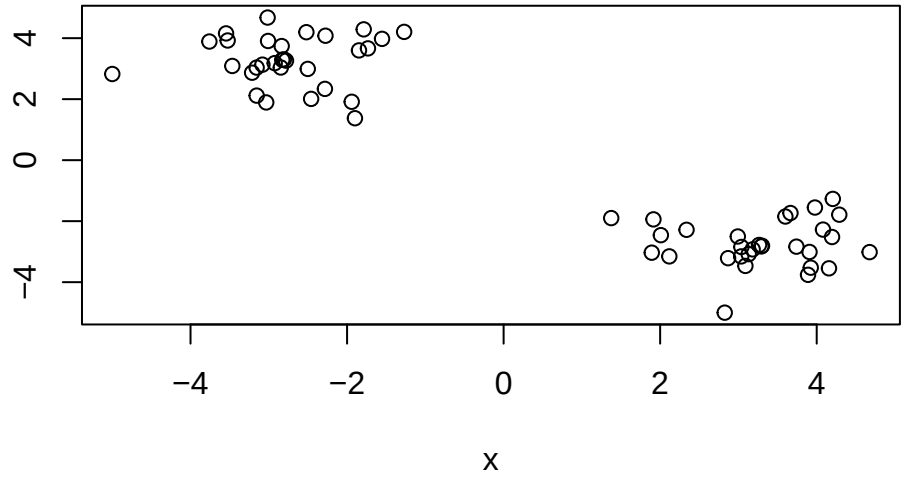
```
 [1] -3.759159 -2.780158 -2.459129 -2.835556 -4.999761 -3.466449 -2.275940
 [8] -1.733053 -3.525540 -1.787455 -3.033048 -3.213019 -1.899389 -3.154920
[15] -3.155554 -2.827863 -3.081583 -2.519829 -1.270799 -3.009253 -2.806691
[22] -1.847785 -2.846435 -2.924525 -3.015139 -1.940719 -2.502004 -1.553212
[29] -2.282888 -3.546160  4.156806  2.337124  3.977242  2.991963  1.914273
[36]  4.676036  3.181637  3.037712  3.599684  3.301369  3.908077  4.205901
[43]  4.196089  3.131311  3.291763  3.035339  2.116983  1.374766  2.867911
[50]  1.892864  4.288538  3.924251  3.664874  4.080553  3.088928  2.825037
[57]  3.740824  2.008975  3.262943  3.889383
```

```
rev(x)
```

```
 [1]  3.889383  3.262943  2.008975  3.740824  2.825037  3.088928  4.080553
 [8]  3.664874  3.924251  4.288538  1.892864  2.867911  1.374766  2.116983
[15]  3.035339  3.291763  3.131311  4.196089  4.205901  3.908077  3.301369
[22]  3.599684  3.037712  3.181637  4.676036  1.914273  2.991963  3.977242
[29]  2.337124  4.156806 -3.546160 -2.282888 -1.553212 -2.502004 -1.940719
[36] -3.015139 -2.924525 -2.846435 -1.847785 -2.806691 -3.009253 -1.270799
[43] -2.519829 -3.081583 -2.827863 -3.155554 -3.154920 -1.899389 -3.213019
```

```
[50] -3.033048 -1.787455 -3.525540 -1.733053 -2.275940 -3.466449 -4.999761
[57] -2.835556 -2.459129 -2.780158 -3.759159
```

```
z<- cbind(x,rev(x))
plot(z)
```




```
p<- (1:5)

cbind(p,p)
```

```
     p p
[1,] 1 1
[2,] 2 2
[3,] 3 3
[4,] 4 4
[5,] 5 5
```

```
rbind(p,p)
```

```
  [,1] [,2] [,3] [,4] [,5]
p    1    2    3    4    5
p    1    2    3    4    5
```

```
rev(p)
```

```
[1] 5 4 3 2 1
```

Now we can run `K-means()` on this input `z` and take a look at the results.

```
km<- kmeans(z, centers=2)
km
```

```
K-means clustering with 2 clusters of sizes 30, 30

Cluster means:
          x
1  3.265638 -2.735101
2 -2.735101  3.265638

Clustering vector:
 [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Within cluster sum of squares by cluster:
[1] 37.28465 37.28465
 (between_SS / total_SS =  93.5 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"

$class
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What component of your results object details cluster assignment/membership?

```
km$cluster
```

```
 [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

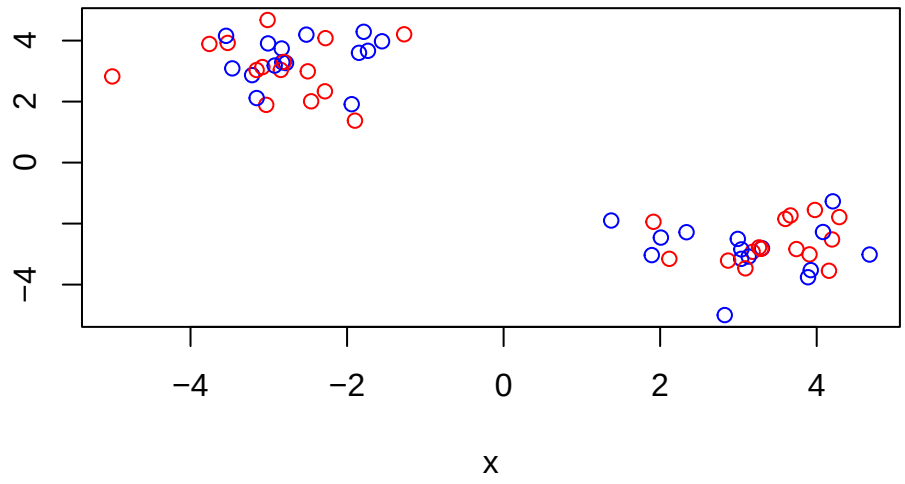
Q. What component of your results object details cluster centers?

```
km$centers
```

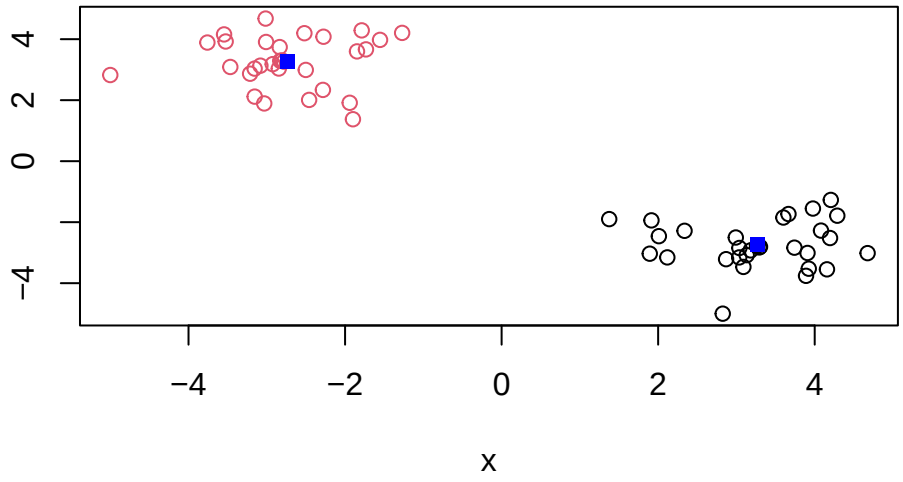
```
          x
1  3.265638 -2.735101
2 -2.735101  3.265638
```

Q. Plot `z` colored by the kmeans cluster assignment and add cluster centers as blue points?

```
plot(z,col=c("red","blue"))
```



```
plot(z,col=km$cluster)
points(km$centers, col="blue", pch=15)
```



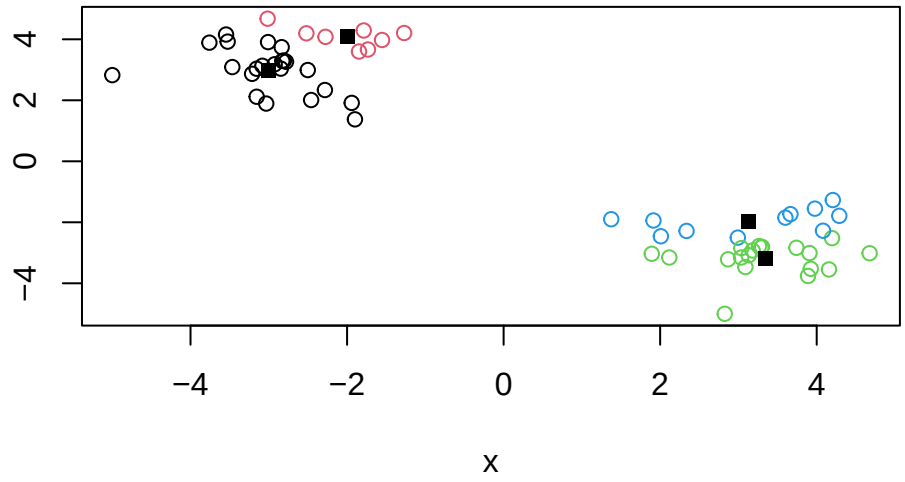
Q. Run a K-means clustering and plot the results asking for 4 clusters (k=4)?

```
km_4<- kmeans(z, centers=4)
km_4
```

```
K-means clustering with 4 clusters of sizes 22, 8, 19, 11

Cluster means:
          x
1 -3.002264  2.967283
2 -2.000402  4.086115
3  3.343435 -3.184244
4  3.131263 -1.959307

Clustering vector:
 [1] 1 1 1 1 1 1 2 2 1 2 1 1 1 1 1 1 1 2 2 1 1 2 1 1 2 1 1 2 1 1 3 4 4 4 4 3 3 3
[39] 4 3 3 4 3 3 3 3 3 4 3 3 4 3 4 4 3 3 3 4 3 3

Within cluster sum of squares by cluster:
```

```
[1] 20.963117  3.089198 13.941319 12.576420
 (between_SS / total_SS =  95.6 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

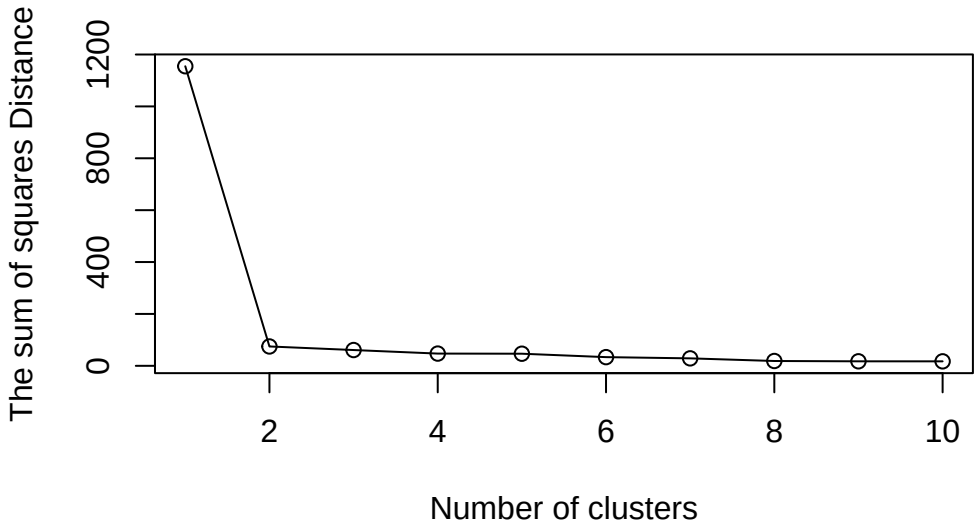
```r
plot(z,col=km_4$cluster)
points(km_4$centers, col="black", pch=15)
```



N.R You need to tell K-means the number of clusters (ie set the `centers=2`)!!

One approach is to try some values for centers and then pick the best one..

```r
ans<- NULL
for(i in 1:10){
km<- kmeans(z, centers=i)
ans <- c(ans,km$tot.withinss)
}

plot(ans, typ="o", xlab= "Number of clusters", ylab="The sum of squares Distance")
```



Hierarchical Clustering

The main function in "base" R for Hierarchical Clustering is called `hclust()`

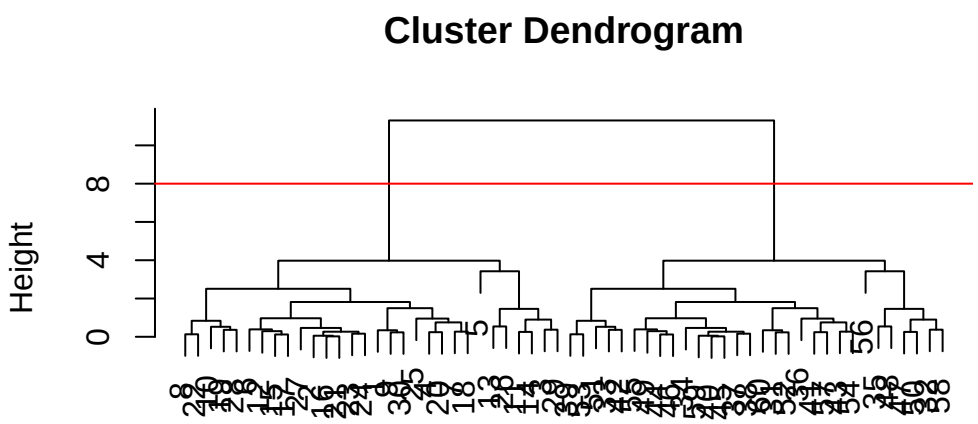
This function does not take your "raw" data for clustering. You must first build a "distance matrix" from your data and pass it as input to `hclust()`

```
d<- dist(z)
hc<- hclust(d)
hc
```

```
Call:
hclust(d = d)

Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

there is a seperate `plot()` for each `hclust()` result object.

```
plot(hc)
abline(h=8, col="red")
```



Once we have the `hclust()` object (the tree of the cluster dendrogram) we can then **cut** the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

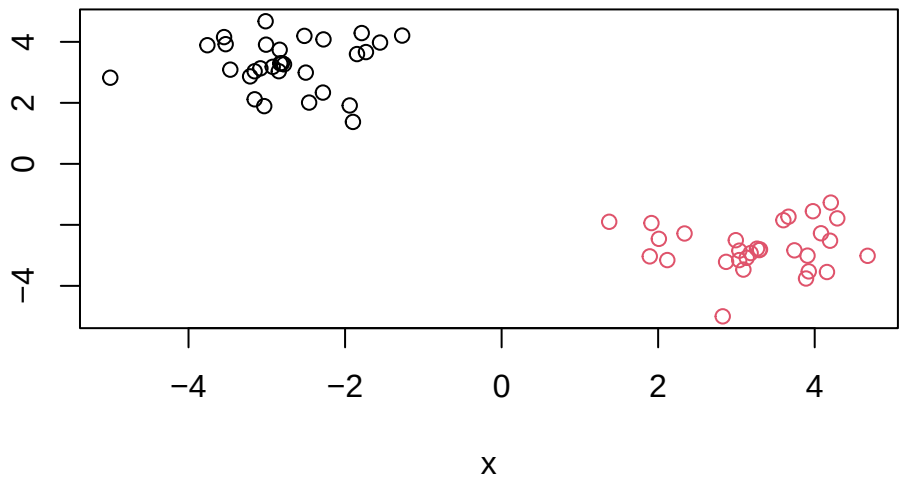
```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

can cut the tree so that we are left with 4 groups (k=4)

```
j <-cutree(hc, k=2)
```

Q. Make a plot of `z` with you `hclust()` results (i.e colored by clustermemberhsip)

```
plot(z, col=j)
```



Principal Component Analysis (PCA)

PCA is a dimensionallity reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

Let's look at some data on the eating habits of people from the UK to see if there are any patterns or trends that have some regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv()` function

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
```

> Q. Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
nrow(x)
```

```
[1] 17
```

```
ncol(x)
```

```
[1] 5
```

```
dim(x)
```

```
[1] 17  5
```

> Q2. Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

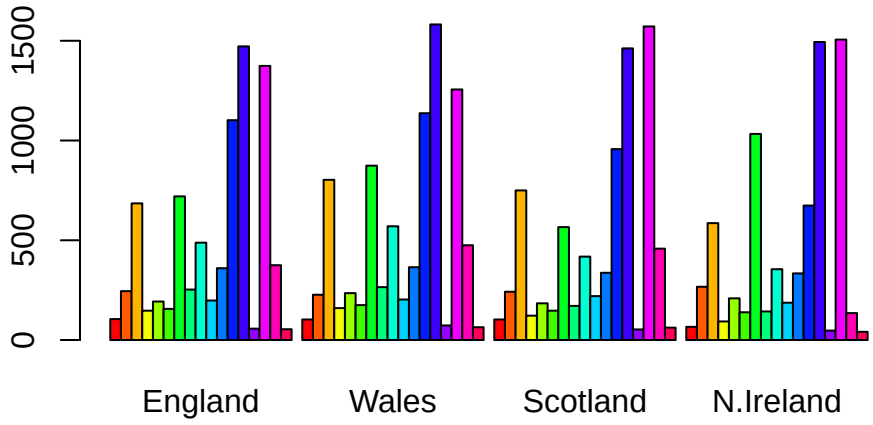
```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url, row.names=1)
```

```
dim(x)
```

```
[1] 17  4
```

The approach of hardcoding into the `read.csv()` function I prefer, as it is more streamlined and robust.The [-1] index method requires more line of code.

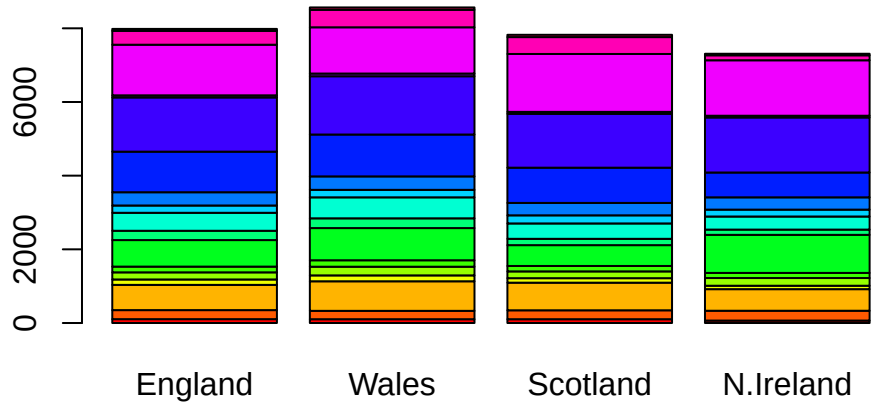
```
# Using base R
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```




```
?barplot()
```

Q3: Changing what optional argument in the above barplot() function results in the following plot?

```
barplot(as.matrix(x), beside=FALSE, col=rainbow(nrow(x)))
```



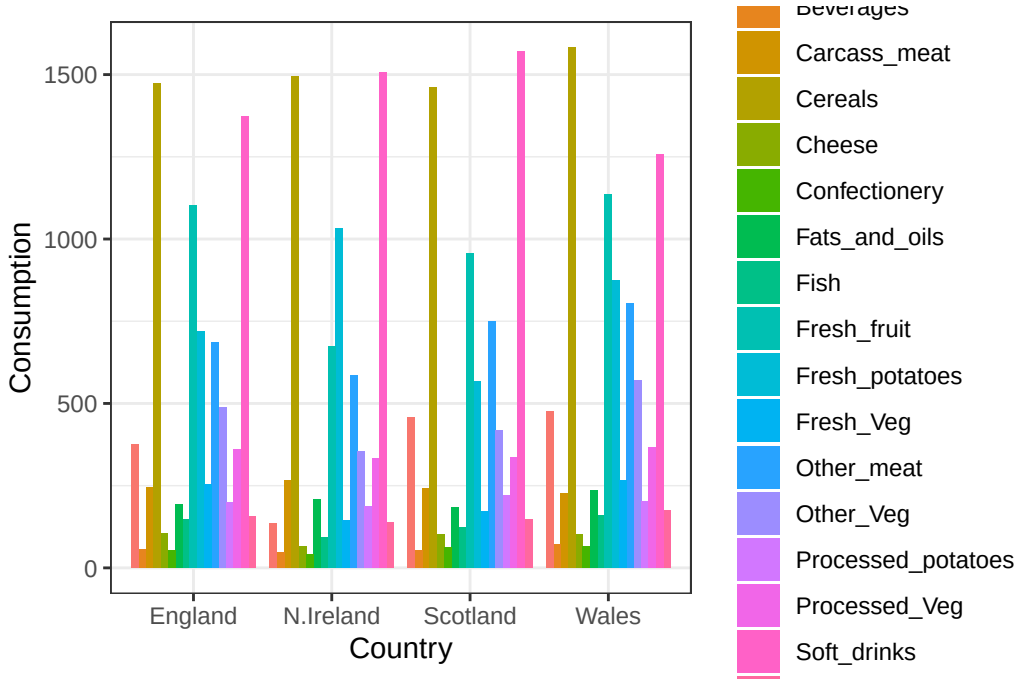
By changing the argument inside `barplot()` –> beside from true to false will change whether the bars are stacked or juxtaposed.

```
library(ggplot2)
```

```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
          tibble::rownames_to_column("Food") |>
          pivot_longer(cols = -Food,
                       names_to = "Country",
                       values_to = "Consumption")

dim(x_long)
```

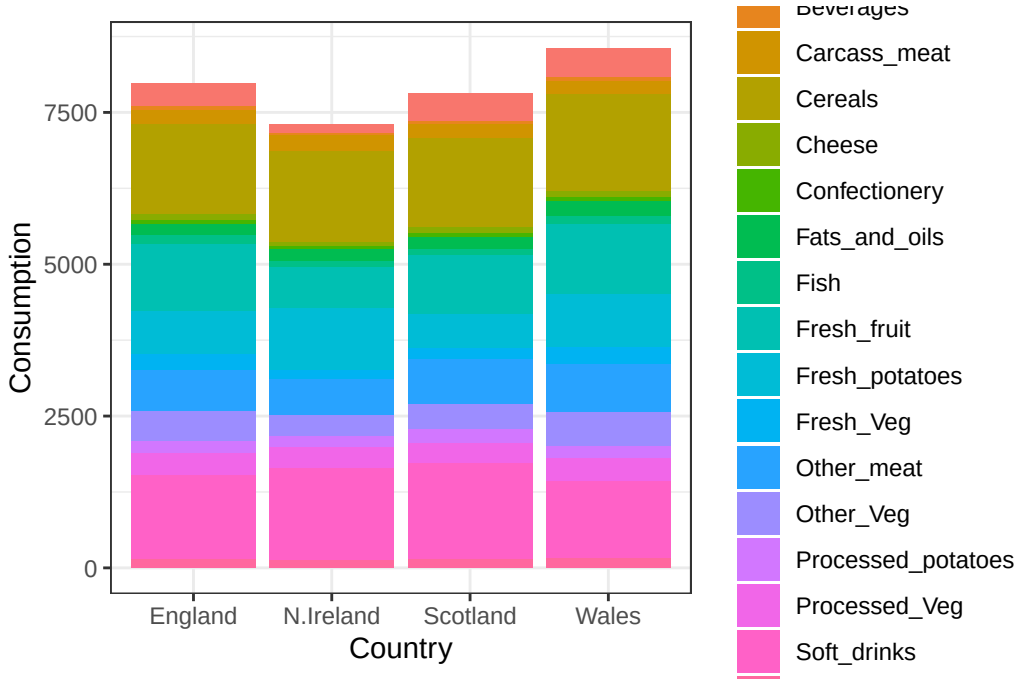
```
[1] 68  3
```

```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

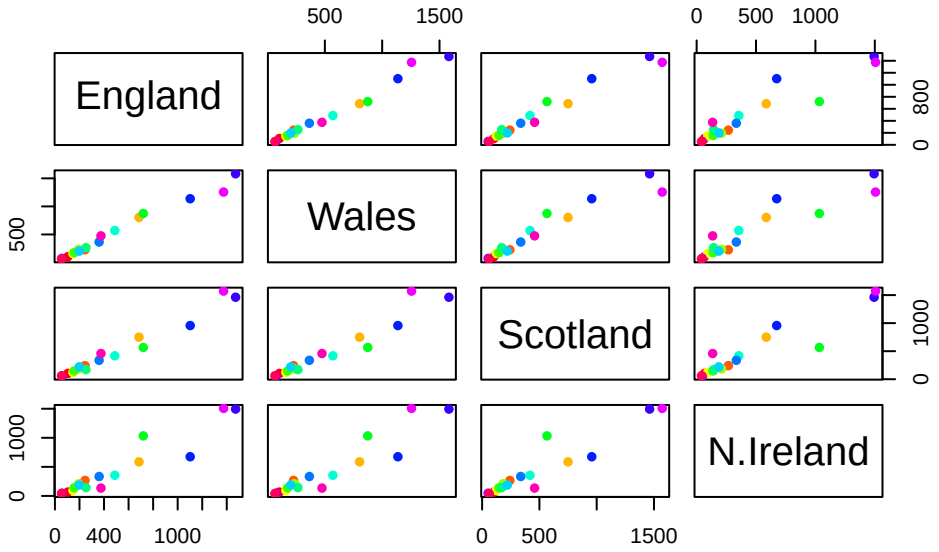
```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col() +
  theme_bw()
```



Via deletion of the position:'dodge' in the `geom_col()` layer of `ggplot()`, this will change the columns into a stacked manner rather than juxtaposed.

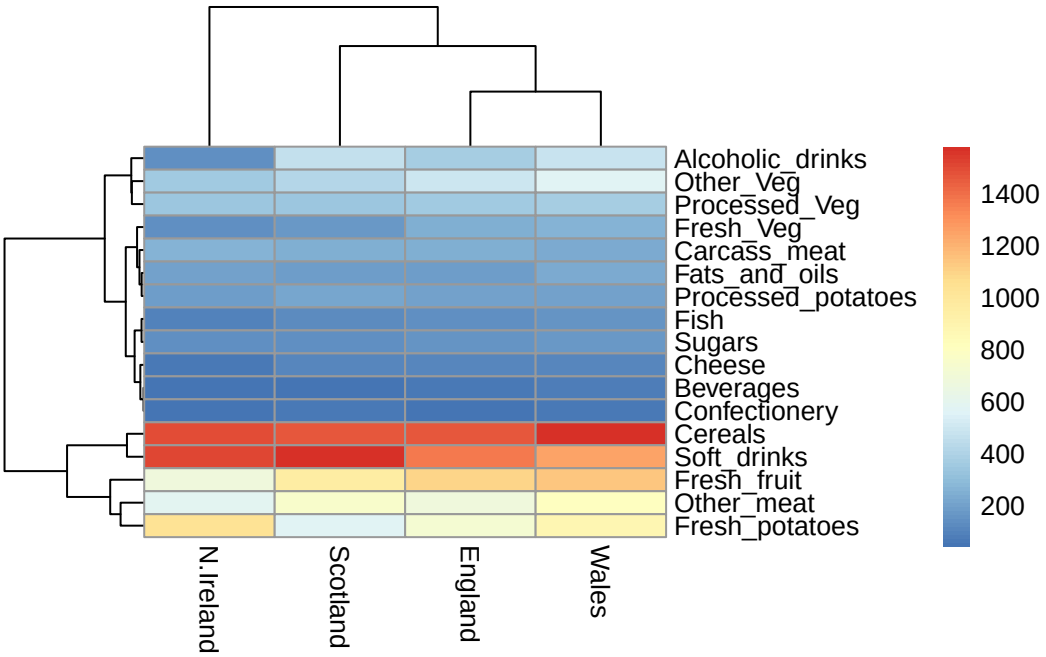
> Q5: We can use the pairs() function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



If a data point lies on a diagonal line, this would indicate that within each country they ingest the same amount of that particular food. If it is outlier depending on the direction there is a discrepancy between the two countries.

```
library(pheatmap)

pheatmap( as.matrix(x))
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

KEY POINT can still be challenging to intrepret!

PCA to the rescue

The main function in "base" R for PCA is called `prcomp()`. This function wants wants the *observations* to be rows and the *variables* to columns. Need to transpose our data, use `t()`.

```
pca<- prcomp(t(x))
summary(pca)
```

```
Importance of components:
                            PC1      PC2      PC3       PC4
Standard deviation     324.1502 212.7478 73.87622 2.921e-14
Proportion of Variance   0.6744   0.2905  0.03503 0.000e+00
Cumulative Proportion    0.6744   0.9650  1.00000 1.000e+00
```

The return `pca` object has components that we can use to make our main results figures:

```
attributes(pca)
```

```
$names
[1] "sdev"     "rotation" "center"   "scale"    "x"

$class
[1] "prcomp"
```

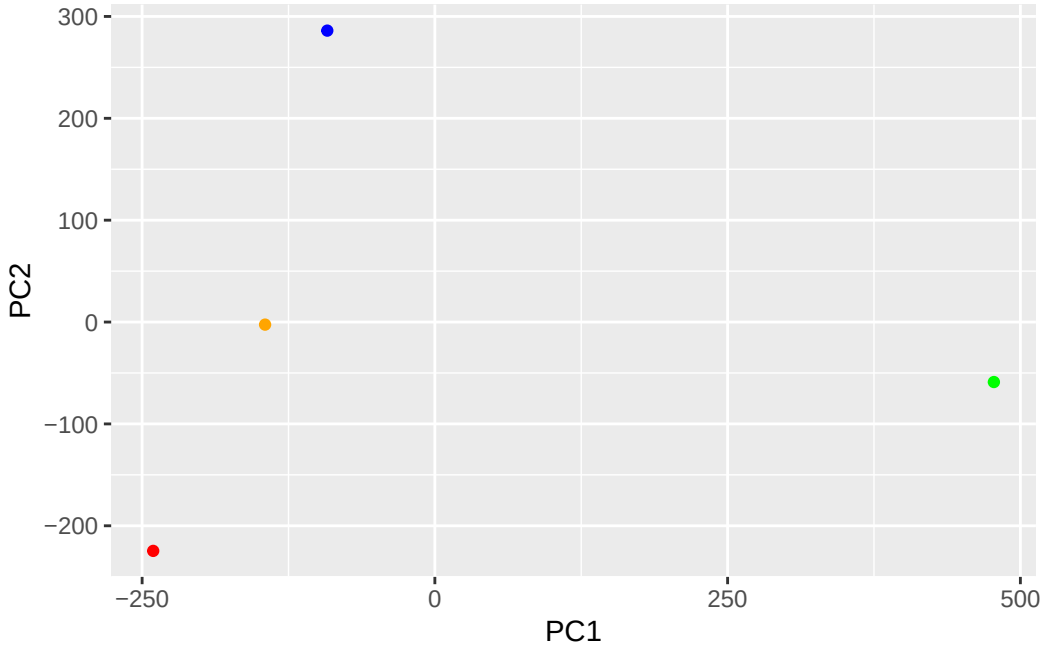
The main result figure form this analysis is called a **"PC score plot"** or "ordination plot" "PC plot" or PC1 versus PC2 plot.

This plot shows how samples (in this case countries) relate to each other along our new PC axis.

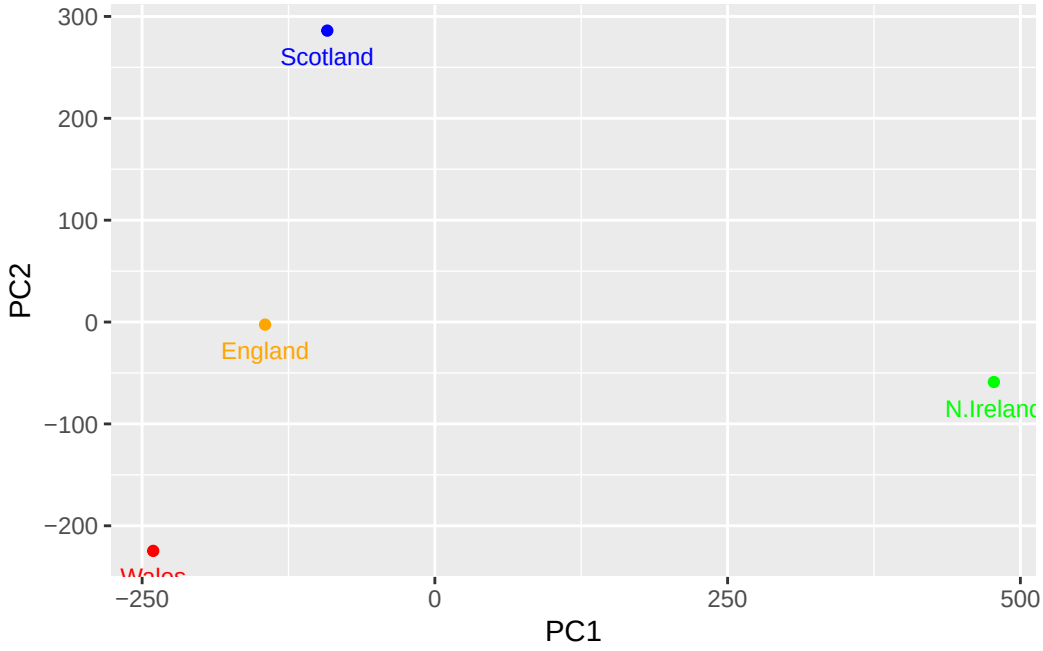
```
mycols<- c("orange", "red", "blue", "green")
pca$x
```

```
                PC1         PC2        PC3           PC4
England   -144.99315   -2.532999 105.768945 -9.152022e-15
Wales     -240.52915 -224.646925 -56.475555  5.560040e-13
Scotland   -91.86934  286.081786 -44.415495 -6.638419e-13
N.Ireland  477.39164  -58.901862  -4.877895  1.329771e-13
```

```
ggplot(pca$x)+
  aes(PC1, PC2)+
geom_point(colour = mycols)
```



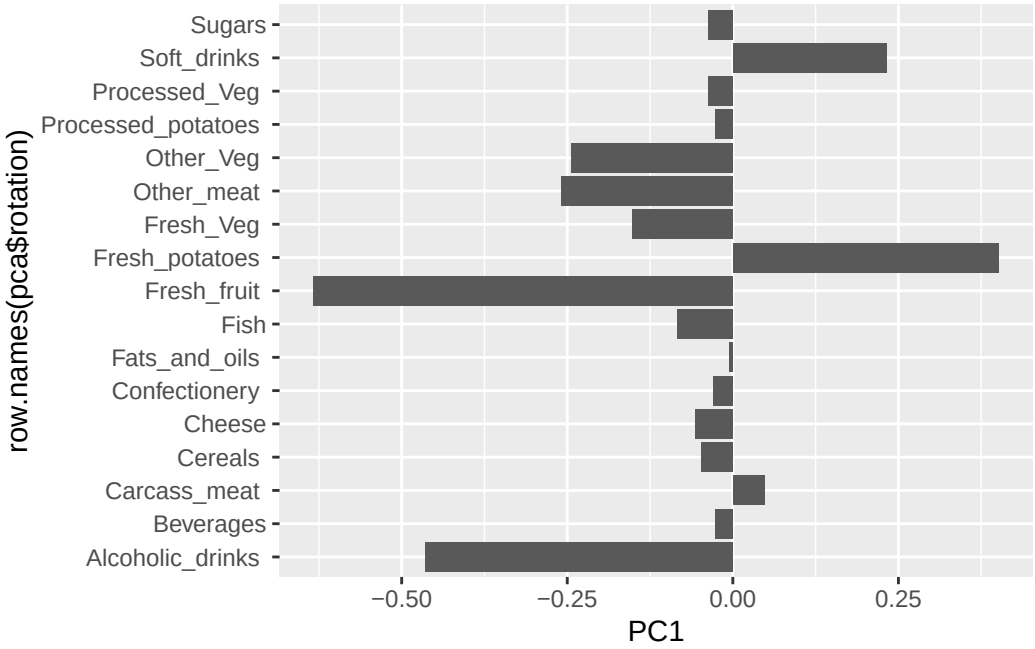

```
ggplot(pca$x)+
  aes(PC1, PC2, label=row.names(pca$x))+
geom_point(col=mycols)+
  geom_text(size=3,vjust=2, col=mycols)
```




```
pca$rotation
```

	PC1	PC2	PC3	PC4
Cheese	-0.056955380	0.016012850	0.02394295	-0.409382587
Carcass_meat	0.047927628	0.013915823	0.06367111	0.729481922
Other_meat	-0.258916658	-0.015331138	-0.55384854	0.331001134
Fish	-0.084414983	-0.050754947	0.03906481	0.022375878
Fats_and_oils	-0.005193623	-0.095388656	-0.12522257	0.034512161
Sugars	-0.037620983	-0.043021699	-0.03605745	0.024943337
Fresh_potatoes	0.401402060	-0.715017078	-0.20668248	0.021396007
Fresh_Veg	-0.151849942	-0.144900268	0.21382237	0.001606882
Other_Veg	-0.243593729	-0.225450923	-0.05332841	0.031153231
Processed_potatoes	-0.026886233	0.042850761	-0.07364902	-0.017379680
Processed_Veg	-0.036488269	-0.045451802	0.05289191	0.021250980
Fresh_fruit	-0.632640898	-0.177740743	0.40012865	0.227657348
Cereals	-0.047702858	-0.212599678	-0.35884921	0.100043319
Beverages	-0.026187756	-0.030560542	-0.04135860	-0.018382072
Soft_drinks	0.232244140	0.555124311	-0.16942648	0.222319484
Alcoholic_drinks	-0.463968168	0.113536523	-0.49858320	-0.273126013
Confectionery	-0.029650201	0.005949921	-0.05232164	0.001890737

```
#how the individual variables contribute to the PCA
ggplot(pca$rotation)+
  aes(PC1, row.names(pca$rotation))+
  geom_col()
```



The loading plot will show you how the different countries (variables) are different and in what manner they are different. For example N Ireland is highly different than the other 3 in terms of PCA 1. The score plot (second)shows how the variables (food) play a role in the PCA scores, for example in Northern Ireland it is shown that they ingest more potatoes and soft drinks than the other three, because it have a more positive PCA 1 score.